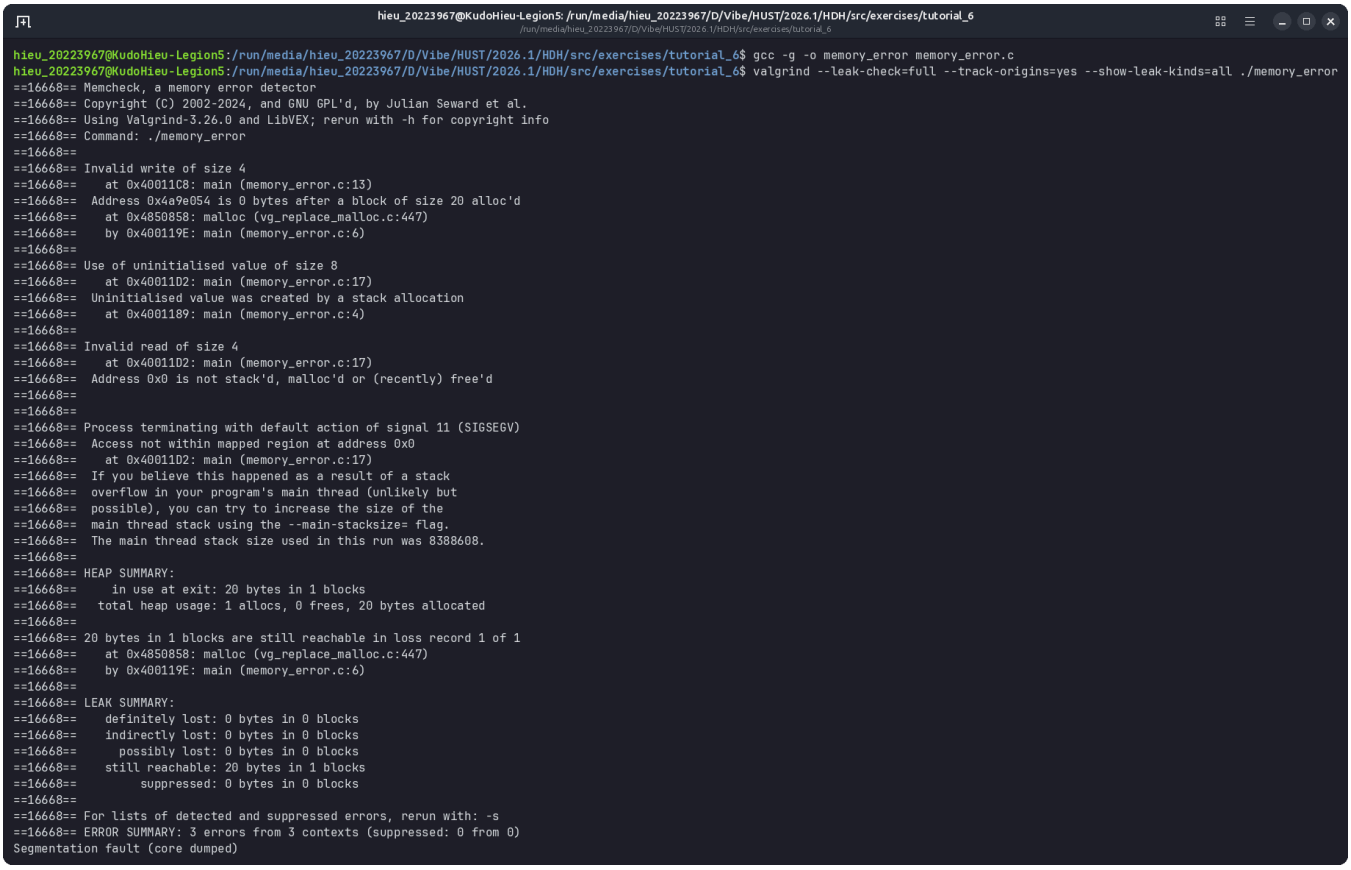
Tutorial 6 – Memory Profiling with Valgrind

Sinh viên: Mai Duy Hiếu · **MSSV:** 20223967
Môi trường: Ubuntu 26.04 (x86-64), gcc 15.2.0, Valgrind 3.26.0 (Memcheck, Massif)

1. Chạy Valgrind với chương trình gốc `memory_error.c`

```
gcc -g -o memory_error memory_error.c
valgrind --leak-check=full --track-origins=yes --show-leak-kinds=all ./memory_error
```

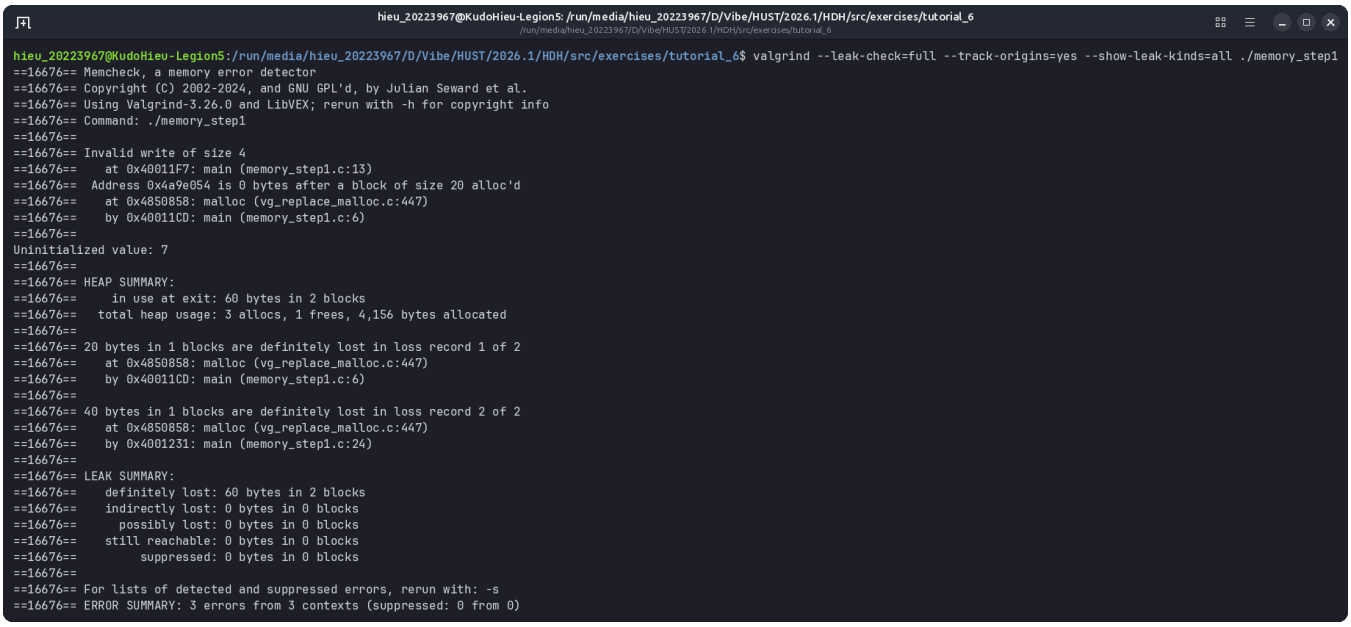


Valgrind báo	Dòng	Nguyên nhân
`Invalid write of size 4` — *0 bytes after a block of size 20*	13	`ptr1[5] = 42`: khối 5 × 4 = 20 byte chỉ có chỉ số 0..4, nên ghi ngay sát sau khối.
`Use of uninitialised value of size 8` — *created by a stack allocation*	17	Dùng giá trị của con trỏ `ptr2` (8 byte trên x86-64) khi chưa được gán.
`Invalid read of size 4` — *Address 0x0*, rồi `SIGSEGV`	17	Giá trị rác của `ptr2` tình cờ là 0, nên `*ptr2` đọc địa chỉ NULL và chương trình bị **Segmentation fault**.

> **Khác với output mẫu trong đề:** chương trình bị crash ở dòng 17, nên dòng 23 (`malloc` cho `ptr3`) không được thực thi. Lúc crash, `ptr1` vẫn còn giữ địa chỉ khối 20 byte, nên Valgrind chỉ báo **20 bytes still reachable**, không phải *60 bytes definitely lost* như trong đề. Ngoài ra trên máy 64 bit, con trỏ có kích thước 8 byte (đề ghi *size 4*).

2. Bước trung gian: chỉ sửa `ptr2` (`memory_step1.c`)

Để thấy đủ lỗi rò rỉ bộ nhớ, em sửa riêng lỗi gây crash trước: cho `ptr2` trỏ vào một biến hợp lệ. Khi đó chương trình chạy hết, và Valgrind báo đúng như đề: vẫn còn lỗi ghi ngoài mảng ở dòng 13, và **60 bytes definitely lost in 2 blocks** (20 byte của `ptr1` và 40 byte của `ptr3`).



3. Chương trình đã sửa `memory_fixed.c`

```
/*
 * memory_fixed.c - Tutorial 6: bản đã sửa của memory_error.c
 *
 * Ba lỗi Valgrind (Memcheck) phát hiện trong bản gốc và cách sửa:
 *   [SỬA 1] Ghi ngoài mảng   : ptr1[5] = 42      -> chỉ ghi trong chỉ số 0..4
 *   [SỬA 2] Con trỏ chưa khởi tạo: int *ptr2; *ptr2 -> cấp phát, kiểm tra, gán giá trị trước
khi đọc
 *   [SỬA 3] Rò rỉ bộ nhớ     : quên free(ptr1), free(ptr3) -> free mọi khối đã malloc
 *
 * Biên dịch: gcc -g -Wall -Wextra -o memory_fixed memory_fixed.c
 * Kiểm tra : valgrind --leak-check=full --track-origins=yes --show-leak-kinds=all
./memory_fixed
 */
#include <stdio.h>
#include <stdlib.h>

#define N1 5  /* số phần tử của ptr1 */
#define N3 10 /* số phần tử của ptr3 */

int main(void) {
  // Allocate memory for 5 integers
  int *ptr1 = malloc(N1 * sizeof(int));
  if (ptr1 == NULL) {
    printf("Memory allocation failed!\n");
    return 1;
  }

  // [SỬA 1] Mảng N1 = 5 phần tử chỉ có chỉ số 0..4, nên phần tử cuối là ptr1[N1 - 1].
  // Khởi tạo cả mảng để không phần tử nào bị đọc khi còn chứa giá trị rác.
  for (int i = 0; i < N1; i++)
    ptr1[i] = 0;
```

```
  ptr1[N1 - 1] = 42;
  printf("ptr1[%d] = %d\n", N1 - 1, ptr1[N1 - 1]);

  // [SỬA 2] Con trỏ phải trỏ tới vùng nhớ hợp lệ trước khi dereference (*ptr2).
  // Cấp phát 1 int, kiểm tra malloc thành công, rồi GÁN giá trị trước khi đọc.
  int *ptr2 = malloc(sizeof(int));
  if (ptr2 == NULL) {
    printf("Memory allocation failed!\n");
    free(ptr1); // đã cấp ptr1 thì thoát sớm cũng phải trả lại
    return 1;
  }
  *ptr2 = 0;
  printf("Initialized value: %d\n", *ptr2);

  // Allocate additional memory for another pointer
  int *ptr3 = malloc(N3 * sizeof(int));
  if (ptr3 == NULL) {
    printf("Memory allocation failed!\n");
    free(ptr1);
    free(ptr2);
    return 1;
  }

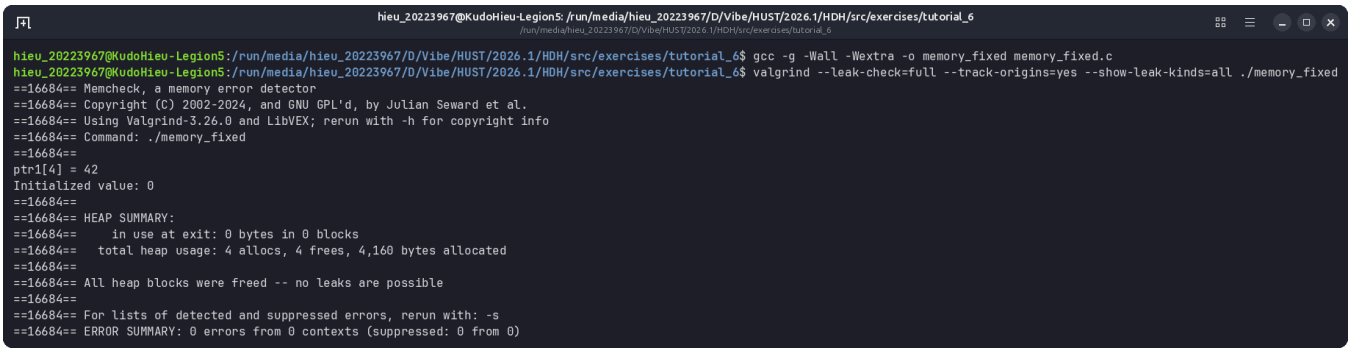
  // [SỬA 3] Mỗi malloc phải có đúng một free tương ứng khi không dùng nữa.
  // Gán NULL sau free để nếu lỡ dùng lại thì lỗi lộ ra ngay (không thành dangling pointer).
  free(ptr1);
  ptr1 = NULL;
  free(ptr2);
  ptr2 = NULL;
  free(ptr3);
  ptr3 = NULL;

  return 0;
}
```

4. Giải thích các dòng code thêm / sửa

#	Lỗi	Code gốc	Code đã sửa	Giải thích
1	Ghi ngoài mảng	`ptr1[5] = 42;`	`for (i = 0; i < N1; i++) ptr1[i] = 0;` `ptr1[N1 - 1] = 42;`	Mảng 5 phần tử có chỉ số 0..4, nên phần tử cuối là `ptr1[4]`. Dùng hằng `N1` để kích thước cấp phát và chỉ số luôn khớp nhau. Khởi tạo cả mảng để không phần tử nào bị đọc khi còn chứa giá trị rác.
2	Con trỏ chưa khởi tạo	`int *ptr2;` `printf(..., *ptr2);`	`int *ptr2 = malloc(sizeof(int));` `if (ptr2 == NULL) { ... }` `*ptr2 = 0;`	Con trỏ phải trỏ tới vùng nhớ hợp lệ trước khi dereference. Cần khởi tạo cả **con trỏ** (cấp phát và kiểm tra `malloc` thành công) lẫn **ô nhớ** nó trỏ tới (gán giá trị) trước khi đọc.
3	Rò rỉ bộ nhớ	không có `free`	`free(ptr1); ptr1 = NULL;` `free(ptr2); ptr2 = NULL;` `free(ptr3); ptr3 = NULL;`	Mỗi `malloc` có đúng một `free` tương ứng. Gán `NULL` sau khi giải phóng để tránh *dangling pointer*: nếu lỡ dùng lại thì lỗi lộ ra ngay.
+	Rò rỉ khi thoát sớm	—	Trong các nhánh `if (... == NULL)`: `free` các khối đã cấp rồi mới `return 1`	Nếu `malloc` thứ hai hoặc thứ ba thất bại thì các khối đã cấp trước đó vẫn được giải phóng. Bản sửa mẫu trong đề chưa kiểm tra `malloc` của `ptr2` và `ptr3`.
+	Cảnh báo khi biên dịch	`gcc -g`	`gcc -g -Wall -Wextra`	Với bản gốc, gcc đã cảnh báo trước `'ptr2' may be used uninitialized` và `unused variable 'ptr3'`. Bản đã sửa biên dịch không còn cảnh báo nào.

5. Kết quả Valgrind sau khi sửa



> **ERROR SUMMARY: 0 errors · All heap blocks were freed — no leaks are possible.**
> Valgrind đếm 4 lần cấp phát trong khi code gọi `malloc` 3 lần. Lần thứ 4 là bộ đệm 4096 byte mà `printf` tự cấp cho `stdout`, và glibc giải phóng nó khi chương trình kết thúc (4160 = 20 + 4 + 40 + 4096 byte). Đo bằng Massif, đỉnh heap của chương trình là khoảng 4.1 KB.

6. Kết luận

- Valgrind Memcheck phát hiện đủ 3 loại lỗi trong đề: ghi ngoài vùng cấp phát, dùng giá trị chưa khởi tạo, rò rỉ bộ nhớ. Nhờ biên dịch với `-g`, mỗi lỗi được chỉ ra đúng file và số dòng.
- Valgrind chỉ kiểm tra những đoạn code **thực sự chạy**. Ở bản gốc, chương trình crash trước dòng 23 nên rò rỉ của `ptr3` chưa lộ ra. Vì vậy em sửa từng lỗi rồi chạy lại Valgrind sau mỗi bước.
- Bản đã sửa không còn lỗi và không rò rỉ bộ nhớ.

File nộp kèm: `memory_error.c` (bản gốc của đề), `memory_step1.c` (bước trung gian), `memory_fixed.c` (bản đã sửa), thư mục `screenshots/` (ảnh kết quả Valgrind).